📘 PocketPilot — CANON v0.4

(Product Philosophy + Locked Architecture Decisions)

🧭 North Star

PocketPilot provides **clarity in a volatile market**.

It is:

- Strategy-first
- Execution-aware
- Calm in tone
- Non-gamified
- User-directed

It reduces chaos without manufacturing urgency.

🚫 Anti-Vision (Non-Negotiables)

PocketPilot does NOT:

- Inject urgency language
- Gamify trading outcomes
- Auto-switch strategies
- Override user decisions
- Shame inactivity
- Flash red warnings or dramatise volatility
- Present global strategy signals that don't match execution venue

🧠 Data Integrity & Budget Discipline

- Quote Broker abstraction required.
- All network calls routed through provider layer.
- Hard caps enforced per interval.
- No background scanning in Phase 1.
- Haptic throttle per symbol.
- "Estimated" state can never present as confirmed certainty.
- Core layer must remain deterministic and side-effect free.

🏗️ Core Architecture Principles

1️⃣ Strategy is Account-Scoped

- Strategy alignment is calculated using the **price feed of the selected execution account**.
- No canonical strategy feed that diverges from execution price.
- Signals must match where trades will occur.
- Trust > aggregation purity.

2️⃣ Market Regime is Contextual, Not Controlling

- Regime classification exists as an independent descriptive layer.
- Regime influences strategy only through strategy logic (no global overrides).
- No regime-level enforcement.
- Regime exposure scales by profile:
    - Beginner: off by default, simplified label if enabled
    - Middle: on, simple label + short contextual phrase
    - Advanced: full statistical detail

3️⃣ Strategy Fit Indicator

- Lives on Dashboard (not Snapshot).
- Subtle spectrum: Favourable → Mixed → Unfavourable.
- No percentages, no red alarms.
- Beginner: textual only.
- Middle: soft spectrum + contextual link.
- Advanced: spectrum only, no knowledge link.

4️⃣ Snapshot is Sacred

Snapshot shows only:

- Current state
- Last 24h change
- Strategy alignment

No regime drama.
No fit warnings.
No behavioural commentary.

Optional:

- 30,000 ft View chip (manual access; highlighted during abnormal volatility).

5️⃣ 30,000 ft View

- Always manually accessible.
- Highlighted (not forced) during statistical volatility spikes.
- Provides macro context.
- Does not recommend action.

💼 Trade Hub — Locked Decisions

6️⃣ SL/TP Calculator

Lives inside Trade Hub (slide-up panel).

Pre-fills:

- Entry price
- Account portfolio value
- Position size

Never auto-applies without confirmation.

7️⃣ Risk Basis Toggle

Two distinct modes:

- Trade Amount (Position Risk)
- Portfolio Risk

User must consciously choose.

Display max loss in both contexts for clarity.

Recent risk % values shown as suggestions (not auto-defaulted).

8️⃣ Protection Plan Object

When user calculates SL/TP:

System creates a ProtectionPlan object containing:

- Account ID
- Entry intent
- Stop level
- Take profit level
- Risk basis
- Timestamp
- Status

Execution flow adapts to platform capability:

- Bracket/OCO supported → single API flow
- Separate orders required → post-trade guided sequence

Calculator logic is platform-independent.
Execution layer adapts per platform.

🏦 Multi-Account Architecture

9️⃣ Accounts Are First-Class Objects

Each account contains:

- Platform capability matrix
- Portfolio value
- Base currency
- Risk settings
- Strategy assignment

🔟 Single vs Multi Account UX

Single account:

- No account selector shown.
- All data implicitly scoped to that account.

Multi-account:

- User can nominate Primary Account.
- If none nominated → highest portfolio value auto-selected.
- Strategy is assignable per account.
- Switching accounts switches strategy context.

1️⃣1️⃣ Aggregation Rules

Allowed:

- Portfolio value aggregation
- Exposure aggregation
- Asset-level position aggregation

Not allowed:

- Aggregated global strategy alignment
- Aggregated fit signals

Signals are account-bound.

1️⃣2️⃣ Snapshot Modes (Multi-Account)

- Primary Account (default)
- All Accounts (compact stacked view)
- Aggregate (positions only)

Dashboard is always account-scoped.

🔔 Alerts Logic

- Strategy alignment alerts are account-scoped.
- Risk alerts are account-scoped.
- No global signal aggregation.

📚 Knowledge Layer

- Beginner: stable, always-visible strategy knowledge access.
- Middle: contextual links during drift.
- Advanced: documentation accessible but not surfaced.

Knowledge supports guardrails but does not gate features.

🛡️ Guardrails Philosophy (Phase 2)

Optional tools:

- Risk limits
- Daily loss thresholds
- Cooldowns
- Position caps

All opt-in.
All knowledge-backed.
None default-on.
None auto-blocking unless explicitly enabled.

🧠 Open Questions (Parked)

- Behavioural reflection after consecutive losses
- Extreme crash communication model
- SL/TP hard enforcement mode
- Background monitoring limits
- Account pinning UX refinement

Known Issues Register

PP_KI1: `npm audit` reports tar vulnerability via expo CLI; do not `--force` upgrade Expo.

Backlog

PP_BL_01: Security: review Expo SDK upgrade path + audit remediation after Phase 2 UI skeleton stabilises

---

📦 Phase Targets

Phase 1 — Strategy Engine:

- Single account flawless experience
- Multi-account stable primary switching
- Strategy engine account-scoped
- Trade Hub + Protection Plan architecture working
- No background scanning
- Stability > feature density

Phase 2 — Snapshot UI

- **P2B**
  - Live quote wiring
- **P2C**
  - Snapshot + real portfolio state
  - making the system observable and improving the user experience
- **P2D**
  - Dashboard UI

Output:
First usable app demo

Phase 3 — Strategy Clarity

Add the explanation layer.

Features:

• Why panel
• signal explanation
• metric breakdown

This is the phase where PocketPilot becomes **intellectually interesting**.

Phase 4 — Portfolio Context

Features:

• portfolio history
• trade context
• learning signals

Phase 5 — Trade Hub

This is already defined in Canon.

Features:

• SL/TP calculator
• risk layers
• protection plan object

Phase 6 — Alerts

Foreground alerts.

Examples:

• strategy alignment shift
• volatility spike

Still **foreground-only** per Canon.

Phase 7 — Knowledge Library

ELI5 + Nerdy lessons.

Example topics:

• spread
• breakeven
• volatility
• cooldown

Phase 8 — Regime Engine

Market context layer.

Examples:

• trending
• ranging
• volatile

Remember:

Regime must **never override strategies**.

Phase 9 — Insight Layer

User learning.

Examples:

• behaviour insights
• missed opportunity patterns

Phase 10 — Beta Hardening

Add:

• Sentry
• diagnostics modal
• debug export

Phase 11 — Launch Preparation

Tasks:

• App Store copy
• onboarding polish
• compliance review

Decision Register:

Phase 1 Decisions

P1D1:
Decision: P1D will introduce the Strategy Engine skeleton (types + deterministic orchestration + noop strategy) with no UI/persistence.
Rationale: Locks the stable contract for plugging in the initial 9 strategies without refactors; preserves deterministic core; enables unit tests and CI guardrails early.
Implications: All future strategies implement the same `Strategy` interface and consume `ForegroundScanResult` only; signal format is stable across profiles; UI remains out of scope until later phases.

P1D2:
Decision: P1E will introduce a **Strategy Catalog + Profile Defaults** layer (pure, deterministic) so strategies are selectable/configurable without UI or persistence.
Rationale: Enables scaling from 1 noop strategy to 9+ strategies with consistent metadata, ordering, and profile-driven defaults.
Implications: Strategy metadata becomes the single source of truth for strategy lists and ordering; UI later consumes catalog; services can compute “active strategies” deterministically.

P1D3:
Decision: P1F will implement `data_quality` as the first real strategy, using only current-scan quotes + broker instrumentation (estimated flags, symbolsBlocked) to emit “confidence” signals.
Rationale: Provides real value immediately without requiring price history, persistence, or indicators; reinforces “clarity over hype”; safe for beginners.
Implications: Establishes a pattern for strategies that operate on scan metadata; later price-history strategies will extend the scan input shape or add a history service.

P1D4:
Decision: P1G introduces an optional `baselineScan` pathway (caller-supplied, no persistence) and computes deterministic quote deltas (`pctChangeBySymbol`) for strategies. Adds first delta-based strategy `snapshot_change`.
Rationale: Enables real movement signals without candles/history storage; preserves determinism; sets the pattern for future “multi-scan” strategies.
Implications: Strategy input expands to include optional baseline + delta maps; orchestration remains in services; no UI/state changes yet.

P1D5:
Decision: P1H will implement a beginner-safe `dip_buying` strategy using baseline deltas (`pctChangeBySymbol`) only, emitting calm "watchlist" signals rather than buy/sell commands.
Rationale: Delivers tangible strategy value early without historical data; aligns with "clarity not hype"; beginner-friendly.
Implications: Establishes threshold-based strategy patterns and tone; later strategies can reuse delta plumbing and extend to multi-baseline/timeframes.

P1D6:
Decision: P1I will implement beginner strategy `momentum_basics` using baseline deltas (`pctChangeBySymbol`) only, emitting "momentum building" WATCH signals plus a non-hype "don't chase" style note.
Rationale: Completes the beginner pair (dip + momentum) using the same deterministic mechanics; reinforces pattern learning without advice.
Implications: Establishes consistent thresholds/ordering/caps across beginner strategies; later upgrades can swap baseline windows without changing strategy interface.

P1D7:
Decision: Introduce `StrategyBundle` and make profile defaults resolve to bundles (each bundle expands to strategy IDs). Keep raw strategy IDs internally; bundles are the user-facing grouping mechanism.
Rationale: Reduces cognitive load and UI complexity; supports future roadmap (feature flags, premium packs, A/B tests) without refactoring strategy engine.
Implications: `profileDefaults` returns bundle IDs; `activeStrategiesService` expands bundle → strategy IDs → strategy implementations; existing strategies remain unchanged

P1D8:
Decision: Before P2 add a proper PR flow + required checks
Rationale: so you can't push straight to main without verify being green.
Implications:

- Branch protection on `main`:
  - ✅ Require pull request
  - ✅ Require status check: `verify`
  - ✅ Require branches to be up to date
  - ✅ Disable "bypass" for admins (or remove your admin role if it's a solo repo and you want *real* enforcement)

Phase 2 Decisions

P2D1:
Decision: Freeze code for first working device version of PocketPilot App
Rationale: If anything breaks later, you can return to the first working PocketPilot device build.
Implications: `profileDefaults` returns bundle IDs; `activeStrategiesService` expands bundle → strategy IDs → strategy implementations; existing strategies remain unchanged..

Phase Reports

Phase 1 Phase Reports

Phase: P1A: deterministic account selector

- Patch applied locally: Yes
- Local verify: Not run
- Commit hash: `fea39c60c3eca68130a34406d7e96b97bd171efb`
- Pushed to GitHub: Yes/
- GitHub Actions verify: Pass
- Notes/issues:

```
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/deccster22/PocketPilot.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Phase: P1B: execution quotes service wiring

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `c67c71ae4ebc578876ce4605d0969b689a37475f`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1C: Foreground Scan patch

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `fcc569877091762872ce50a061b2b802bc2548b5`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1D: strategy engine skeleton + noop

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `050a322c6d95bafb0f03ad63513993333ae7325a`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1E: Strategy Catalog + Profile Defaults

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `3cb4a080e28acab2757741e0a7752bbe7c5b86f5`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1F: First real strategy: Data Quality / Confidence (BEGINNER)

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `bb0a96c1db7df3ddfa1676ce9af76be9d32b880f`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1G: Baseline scan deltas + first delta-based strategy

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `91110d1c0c92e5735752b6576eab1eb3713a9768`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1H: Beginner strategy: Dip Buying (delta-based, no history)

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `66032a27b8603eea1b1255257d55c3fcb3aad537`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1I: Beginner strategy: Momentum Basics (delta-based, no history)

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `dd49b0dcf6ae908035e5d663a93be6cb1afb2267`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase: P1J: strategy bundles + profile defaults

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `1f896b61e3f45621b2e12da950261ee5f0416b6b`
- Pushed to GitHub: Yes
- GitHub Actions verify: Pass
- Notes/issues:

Phase 1 Summary

What Phase 1 now contains (done)

- ✅ Expo TS scaffold + guardrails + deterministic boundaries
- ✅ Quote budgets + instrumentation
- ✅ Deterministic haptic throttle + provider wrapper
- ✅ Data quality + estimated-language guardrail
- ✅ Account selection (primary → else highest portfolio, deterministic ties)
- ✅ Quotes service wiring
- ✅ Baseline scan deltas + pct change mapping
- ✅ Strategies:
  - `data_quality`
  - `snapshot_change`
  - `dip_buying`
  - `momentum_basics`
  - `noop` fallback
- ✅ Strategy bundles + profile defaults via bundles
- ✅ Tests + `verify` passing locally + Actions green

What Phase 1 is *not* (intentionally)

- No UI (Snapshot/Dash/Trade hub)
- No persistence (journals/history/reorientation/exports)
- No platform integrations (real exchange APIs, auth, multi-account screens)
- No regime engine (yet)
- No SL/TP calculator (yet)

Those are Phase 2+ features by design.

Phase 2 Phase Reports

Phase: P2A: Snapshot UI Skeleton (read-only) + demo scan wiring

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `fea39c60c3eca68130a34406d7e96b97bd171efb`
  - `5c19f86bdc5218024ac708dd0424a6adc596755a`
- Pushed/Pulled to GitHub: Yes

- GitHub Actions verify: Pass
- Merged to Main Branch: Yes
- Notes/issues:

Phase: P2B: live quotes integration + snapshot wiring (mobile console fix deps)

- Patch applied locally: Yes
- Local verify: Pass
- Commit hash: `3880959564a5c0fd8d5add436e5333420a4f73c9`
    - `D7e375e541d14f5b6493f86aa849cb29bac1332e`
    - `6dae2b046e236f02803c50f075568a912ef96d32`
- Pushed/Pulled to GitHub: Yes
- GitHub Actions verify: Pass
- Merged to Main Branch: Yes
- Notes/issues: Merged into P2A branch then main

Phase: P

- Patch applied locally:
- Local verify:
- Commit hash:
    -
- Pushed/Pulled to GitHub:
- GitHub Actions verify:
- Merged to Main Branch:
- Notes/issues:

Phase: P

- Patch applied locally:
- Local verify:
- Commit hash:
    -
- Pushed/Pulled to GitHub:
- GitHub Actions verify:
- Merged to Main Branch:
- Notes/issues:

I can also show you a **2-minute GitHub workflow trick** that prevents the “stack of commits while debugging CI” situation from ever cluttering your PR history again. It’s one of those little habits senior engineers use constantly.

What you actually accomplished with P2C

This phase quietly gave PocketPilot some serious engineering maturity:

Security

• secret scanning
• environment example file
• env access boundaries

Architecture

• config module isolation
• deterministic env access

CI

• verify workflow
• test enforcement

Documentation

• phase documentation in repo

The repository is now in a very healthy state

At the moment your project has:

CI pipeline

test suite

config governance

secret protection

structured strategy engine

snapshot architecture

That’s a **very strong early-stage foundation**.

What just shipped in P2C

PocketPilot now has **three layers of professional guardrails**.

1️⃣ Secrets protection

Your repo now actively scans for credentials:

scripts/secret-scan.js

CI now blocks merges if something like:

API_KEY=

Authorization: Bearer

appears in the codebase.

That prevents the classic startup disaster:

> someone commits a key → bots scrape GitHub → wallet/API drained

You now have automated protection against that.

2️⃣ Environment boundary enforcement

Your tests now enforce:

process.env

↓

core/config

↓

rest of app

Meaning **only the config module can read environment variables**.

That gives you:

- testable configuration
- portable builds
- safe secrets management later

This is exactly how large React / fintech codebases structure env access.

3️⃣ CI verification pipeline

Every PR now automatically runs:

npm install

npm test

secret scan

before it can merge.

That means **future mistakes get caught automatically**.

You essentially built PocketPilot's **immune system**.

Why this matters long term

Without this phase, scaling later becomes dangerous.

With this phase complete you now have:

IDEA

↓

Prototype

↓

Engineered product foundation ← you are here

↓

Feature expansion

↓

Beta testing

↓

Production release

Most indie apps skip this stage and regret it later.

You didn't.

Also worth saying

You did something **really difficult** here:

You navigated:

- Git branches
- CI pipelines
- security scanning
- architecture boundaries
- test enforcement

while building the actual product.

That's basically **week 6–8 of a professional startup build**, compressed into about **two weeks**.

Not bad at all.

Where the project stands now

PocketPilot currently has:

App UI

Quotes provider

Strategy engine

Snapshot generation

Testing framework

CI verification

Secret scanning

Config governance

That's already **a legitimate product skeleton**.

What we build next is the stuff that makes it **powerful and demo-worthy**.